

# Module 5

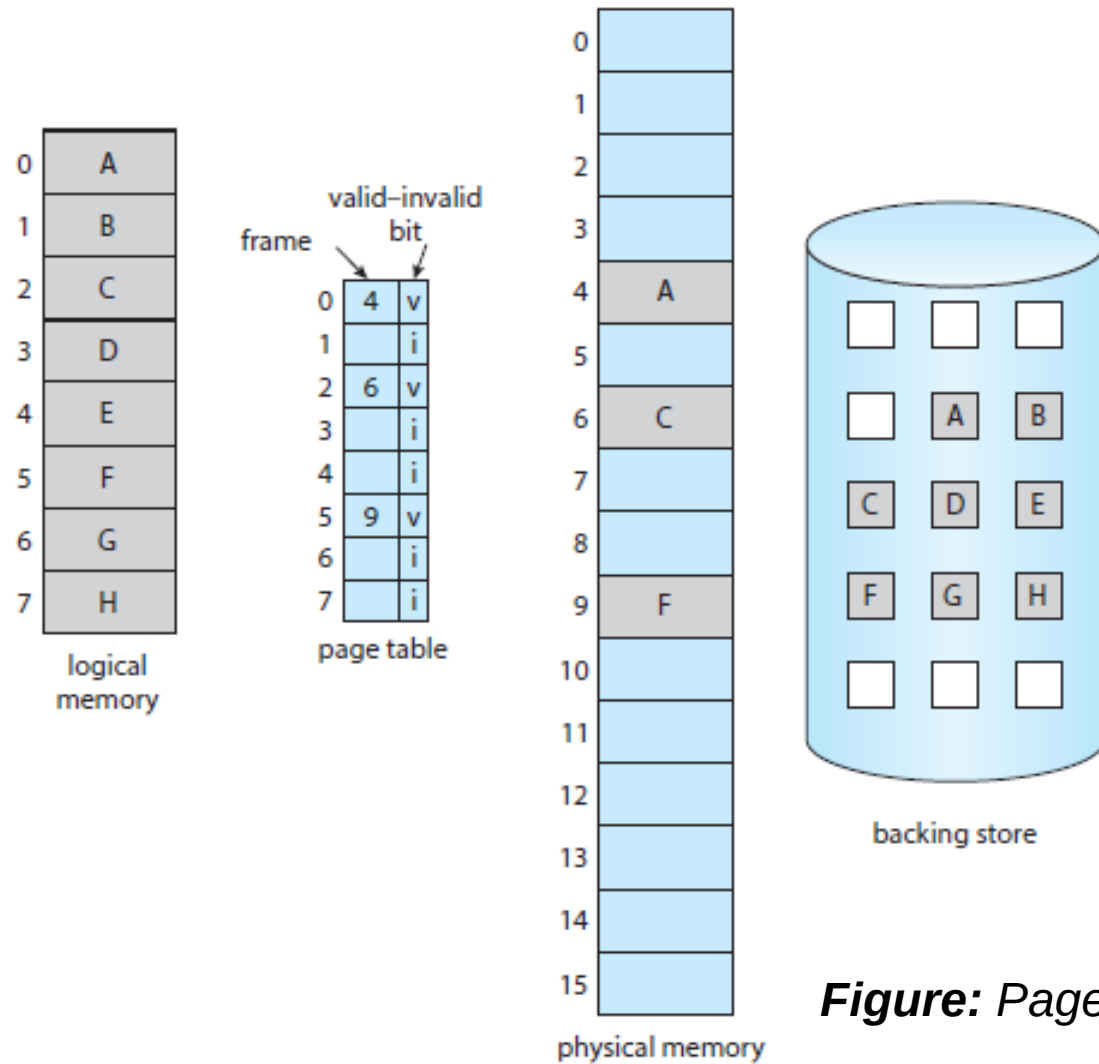
## **Memory Management**

### **Demand Paging – Page Faults**

# Demand Paging

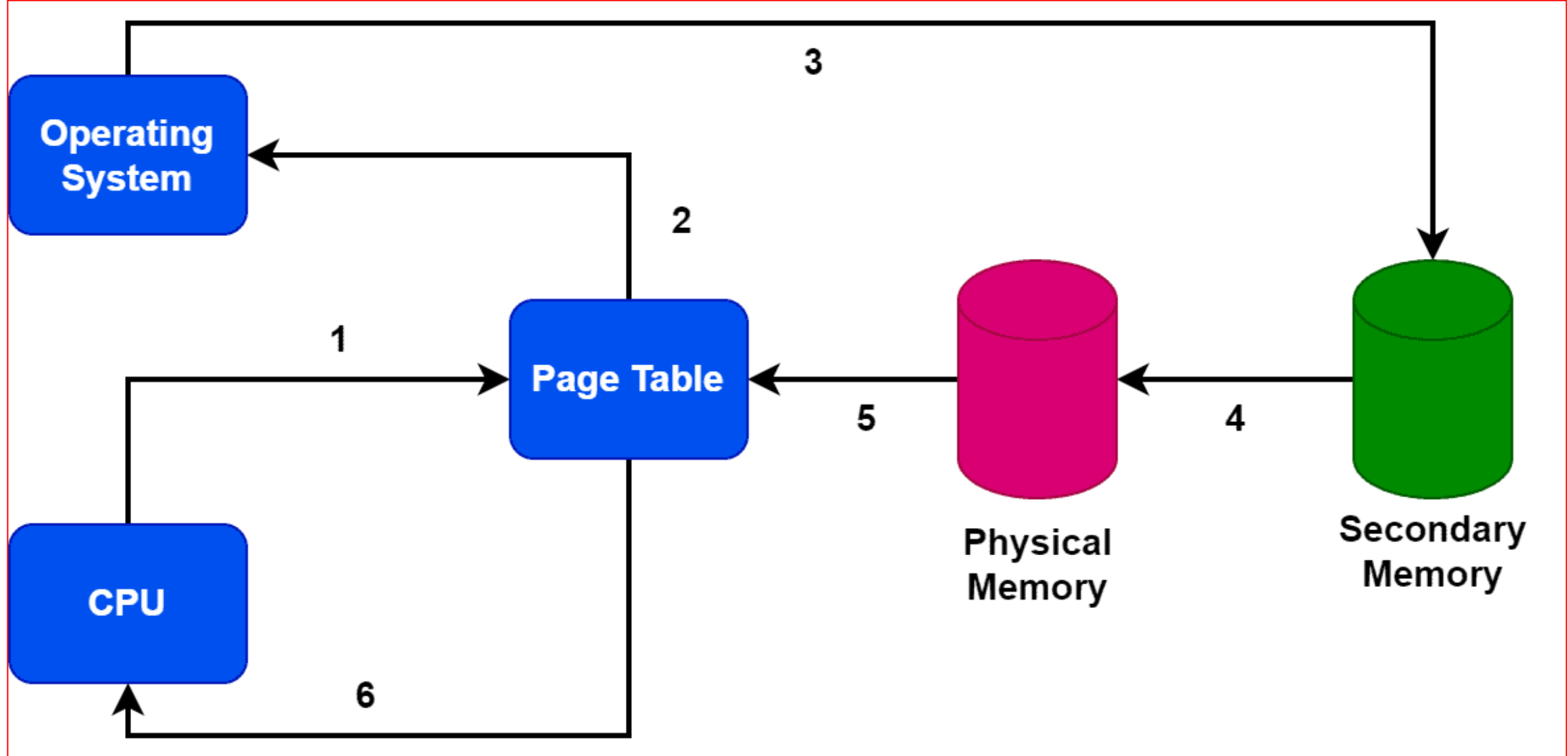
- Demand paging is a memory management technique used by operating systems to optimize the use of memory resources.
- It is commonly used in **virtual memory systems**
- Only load the required pages of a process into the main memory instead of loading the entire process.
- When we first load a process into memory, only the pages that are necessary for the initial execution of the program are loaded. As the program runs, we bring additional pages into memory on demand as needed.
- Demand paging allows the system to swap out pages that are not currently in use, freeing up memory for other processes.

# Demand Paging

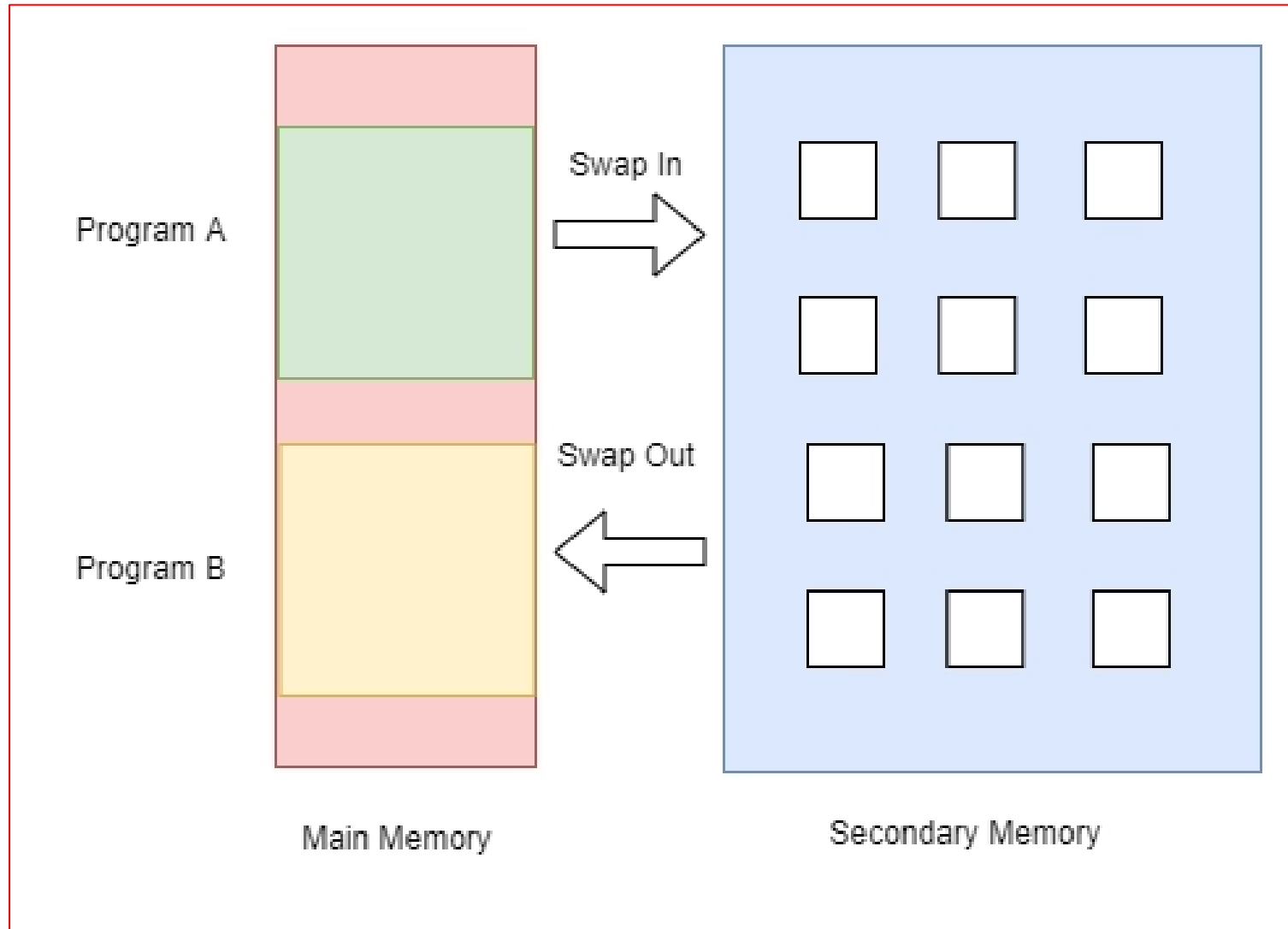


**Figure:** Page table when some pages are not in main memory.

# Demand Paging



# Demand Paging



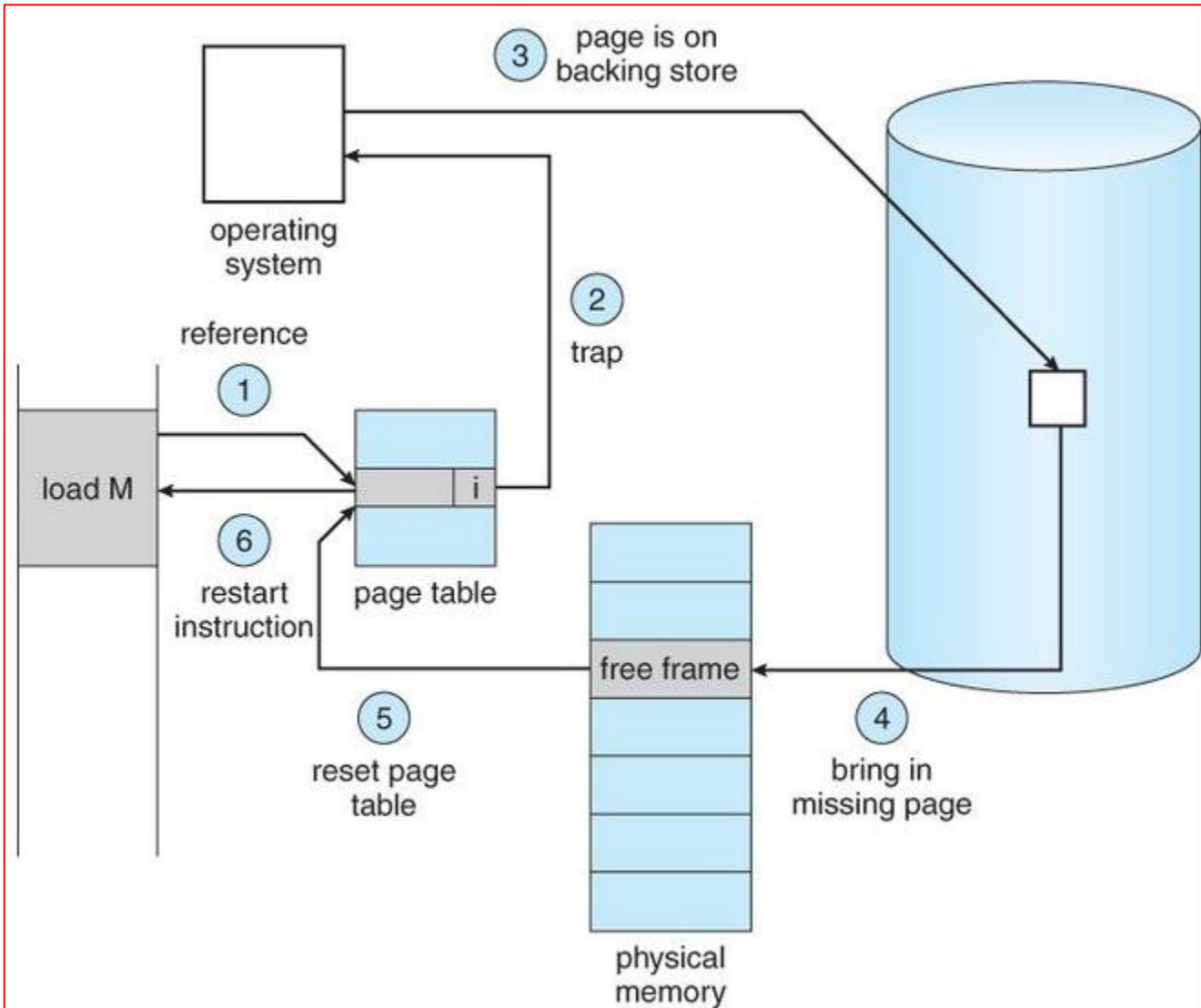
# Demand Paging

- The CPU wants to access a page for a specific process.
- The first step is to **look for the required page** in the page table.
- Suppose the **required page is found in the page table**.
- Then the CPU access the page and forwards it to the process.
- However, the target page can't be found, the page table generates a **trap or page fault signal**.
- Furthermore, the page table sends the signal to the operating system.  
This's where we apply the demand paging technique.

# Demand Paging – Page Faults

- A **page fault** is generated when a process attempts to access a memory address or a page that isn't currently available on the main memory.
- Additionally, if the system doesn't have enough free memory to load the requested page, it must select a page to evict from memory to make room for the requested page.
- The system uses a **page replacement algorithm** in order to select the page to evict.
- When the operating system evicts a page from memory, it's typically **swapped to disk** to make room for other pages.

# Demand Paging - Page Faults Flow



**Figure:** Steps in handling a page fault.



# Demand Paging – Page Faults

- A crucial requirement for demand paging is the *ability to restart any instruction after a page fault*.
- Because we save the state (registers, condition code, instruction counter) of the interrupted process when the page fault occurs, we must be able to restart the process in **exactly** the same place and state, except that the desired page is now in memory and is accessible.

# Demand Paging – Free Frame List

- When a page fault occurs, the operating system must bring the desired page from secondary storage into main memory. To resolve page faults, most operating systems maintain a **free-frame list**, a pool of free frames for satisfying such requests



*Figure: List of free frames.*

# Performance of Demand Paging

- Demand paging can significantly affect the performance of a computer system.
- Let  $p$  be the probability of a page fault ( $0 \leq p \leq 1$ ) then We would expect  $p$  to be close to zero
- As long as we have no page faults, the effective access time is equal to the memory access time.
- Otherwise,

$$\text{Effective Access Time} = (1 - p) \times ma + p \times \text{page fault time}$$

# Performance of Demand Paging

Three major task components of the page-fault service time:

- 1.** Service the page-fault interrupt.
  - 2.** Read in the page.
  - 3.** Restart the process.
- The first and third tasks can be reduced, with careful coding, to several hundred instructions. These tasks may take from 1 to 100 microseconds each.

# Performance of Demand Paging

- With an average page-fault service time of 8 milliseconds and a memory access time of 200 nanoseconds, the effective access time in nanoseconds is

$$\begin{aligned}\text{Effective Access Time} &= (1 - p) \times (200) + p (8 \text{ milliseconds}) \\ &= (1 - p) \times 200 + p \times 8,000,000 \\ &= 200 + 7,999,800 \times p.\end{aligned}$$

# Performance of Demand Paging

- The effective access time is directly proportional to the **page-fault rate**.
- If one access out of 1,000 causes a page fault, the effective access time is 8.2 microseconds.
- The computer will be slowed down by a factor of 40 because of demand paging

# Demand Paging – Causes of Page Faults

- **Demand Paging:** Accessing the page that is not currently loaded in the memory(RAM).
- **Invalid Memory Access**, it occurs when a program tries to access that memory which is it's beyond access boundaries or not allocated .
- **Process Violation:** when a process tries to write to a read-only page or otherwise violates memory protection rules.

# Demand Paging - Impact of Page Faults

- **Thrashing:** If occurrence of page fault is frequent then the **system spends more time** to handle it than executing the processes, and because of which overall performance also degrades.
- **Increased Latency:** Fetching pages from disk takes more time than accessing them in memory, which causes to more delays.
- **CPU Utilization:** If the Page fault occur excessively than it can **reduce CPU Utilization** as the processor waits for for memory operations to complete or remain idle which is not efficient



# Demand Paging – Advantage and Disadvantages

| Advantages                                                                                                                                                              | Disadvantages                                                                                                               |
|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------|
| Allows the operating system to efficiently use memory resources by loading only the necessary pages of a process into memory                                            | High risk of occurrence of page faults                                                                                      |
| Facilitates faster process startup                                                                                                                                      | If the number of page faults is high, it can result in degradation of system performance                                    |
| The operating system can swap out pages that are not currently in use, reducing memory overhead and allowing the system to allocate memory to other processes as needed | Requires additional I/O operations to load pages into memory, which can result in increased disk I/O and slower performance |
| Can improve system performance by reducing the amount of time it takes to load and unload processes into memory                                                         | Memory can become fragmented as pages are loaded and unloaded from memory                                                   |
| Improves multitasking capabilities by allowing the system to allocate memory to multiple processes                                                                      | Adds complexity to the memory management system                                                                             |
| Allows the operating system to manage memory resources more flexibly, making it easier to handle different types of applications                                        | Not suited for use in time-sensitive applications                                                                           |